

변화량·다중 센서와 전처리 파이프라인

들쭉날쭉한 현실 데이터의 시간 축을 바로잡습니다

diff pct_change seasonal_decompose corr
axvspan asfreq interpolate(limit=) pipeline

2026년 8월 28일 (금) · 28일차 · 66차시

교안 21_02 변화량과 주기 분석 · 21_03 다중 센서 상태 변화 분석 · 22_01 측정 주기와 리샘플링 후 결측 · 22_02 누락 시점 처리와 전처리 파이프라인

국립금오공과대학교 컴퓨터공학부 박민호

교안	다룬 범위	상태
21_02 변화량과 주기 분석	diff · 주기성 · 리샘플링 · 시계열 분해 · 실습 1~6	완주
21_03 다중 센서 상태 변화	상관 heatmap · 변화점 · 정상 범위 · 실습 1~5	완주
22_01 측정 주기와 결측	간격 진단 · asfreq · 결측 위치 · 실습 1~6	완주
22_02 누락 처리와 파이프라인	ffill·bfill · 보간 · limit · 함수화 · 실습 1~7	완주
23_01 상태 변화 추적	—	다음 시간

이 책을 읽는 법

배지 · 코드블록 · 이전 회차와의 연결

1. 분류 배지

오늘도 메서드가 대부분입니다. 다만 **옵션(키워드 인자)**이 유난히 많이 나옵니다. `limit=` · `method=` · `center=` 같은 것들인데, 이 옵션 하나가 결과를 크게 바꿉니다.

배지	무엇	판별 신호	오늘의 예
메서드	객체에 붙어 동작하는 함수	점 뒤에 있고 괄호가 있습니다	<code>.diff()</code> <code>.asfreq()</code> <code>.interpolate()</code>
함수	라이브러리가 제공하는 함수	앞에 모듈 이름이 옵니다	<code>seasonal_decompose()</code> <code>plt.axvspan()</code>
키워드	함수에 넘기는 이름 붙은 인자	등호 왼쪽에 옵니다	<code>limit=</code> <code>method=</code> <code>period=</code>
속성	객체가 가진 값	점 뒤에 있고 괄호가 없습니다	<code>df.index</code> <code>result.trend</code>
개념	문법이 아닌 사고 틀	코드로 쓸 수 없습니다	주기성 · 변화점 · 다중공선성 · 파이프라인

개념

오늘 가장 중요한 것은 **옵션 하나가 결론을 바꾼다**는 점입니다. `interpolate()`는 빈 칸을 전부 채우지만 `interpolate(limit=2)`는 **13칸을 일부러 남깁니다**. 그리고 그 13칸이 "여기는 추정하면 안 되는 구간"이라는 신호가 됩니다. **채우는 것이 항상 옳은 것은 아닙니다**.

2. 오늘 쓰는 데이터 세 가지

오늘은 데이터가 세 번 바뀝니다. **성격이 다르기 때문에** 다루는 방법도 달라집니다.

파일	크기	시간 축	특징	쓰는 곳
<code>21_cmapss_fd001_sample.csv</code>	1031행 10열	cycle	엔진 5대 · 결측 0	STEP 1 · 3~4
<code>21_engine1_timestamp_sample.csv</code>	192행 6열	timestamp	1시간 간격 · 빠짐 0	STEP 2
<code>22_열처리.csv</code>	188행 4열	timestamp	간격 3~80초 · 불규칙	STEP 5~7

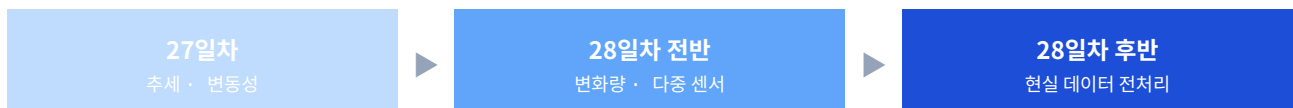
주의

세 번째 데이터가 오늘의 주인공입니다. 앞의 둘은 학습용이라 시간 간격이 완벽하게 일정했지만, 22_열처리.csv는 10초 설정인데 실제 간격이 3초부터 80초까지 제각각입니다.

이것이 현실 데이터의 모습입니다. 통신 지연, 센서 단절, 시스템 부하 때문에 **설정한 대로 기록되지 않습니다**. 그래서 22장 전체가 "분석 전에 시간 축부터 바로잡는 법"에 할애됩니다.

3. 어제와 오늘의 연결

어제는 **깨끗한 데이터로 도구를 익혔습니다**. 오늘은 그 도구를 **더 확장하고**, 마지막에 **지저분한 현실 데이터**에 적용합니다.



어제 배운 것	오늘 확장되는 방향
<code>rolling().mean()</code> — 추세	<code>.diff()</code> 를 더해 매 순간의 발걸음 까지 봅니다
<code>rolling().std()</code> — 변동성	<code>axvspan</code> 으로 의심 구간 을 색칠합니다
센서 하나씩 보기	<code>corr()</code> + <code>heatmap</code> 으로 여러 센서를 함께 봅니다
<code>resample()</code> — 시간 집계	<code>asfreq()</code> 와 나눠 쓰고 불규칙 간격 을 진단합니다
<code>interpolate()</code> — 보간	<code>limit</code> 으로 채우지 말아야 할 구간 을 남깁니다

연결

어제 문서에서 "**추세는 어디로 가는가, 변동성은 얼마나 불안한가**"라고 했습니다. 오늘 여기에 "**변화량은 매 순간 얼마나 움직였는가**"가 더해집니다. 세 도구가 같은 데이터를 세 각도에서 보는 것이고, 21_02 실습 2가 정확히 그 세 관점을 한 화면에 놓습니다.

CONTENTS

목차

오늘 배우는 것을 한눈에

장	제목	무엇을 배우나
CH 0	오늘의 지도	세 관점 · 학습용과 현실 데이터의 차이
STEP 1	변화량으로 급변 잡기	diff · pct_change · 급변 해석 주의 · 실습 1·2
STEP 2	주기성과 시계열 분해	주기 개념 · resample 복습 · decompose · 실습 3~6
STEP 3	센서 간 관계 보기	corr · heatmap · 다중공선성 · 실습 1·2
STEP 4	변화점과 이상 구간	axvline · axvspan · 정상 범위 · 실습 3~5
STEP 5	측정 주기 진단	간격 diff · value_counts · 불규칙의 위험 · 실습 1·2
STEP 6	asfreq로 격자 정규화	asfreq vs resample · 결측 위치·비율 · 실습 3~6
STEP 7	보간과 전처리 파이프라인	ffill·bfill · time 보간 · limit · 함수화 · 실습 1~7
부록 A	치트시트	오늘 나온 모든 도구를 한 표에
부록 B	오류·함정 사전	자주 걸리는 함정 · 결론이 틀리는 경우
부록 C	실습 하이라이트와 다음 시간	핵심 결론 정리 · 23장 예고

개념

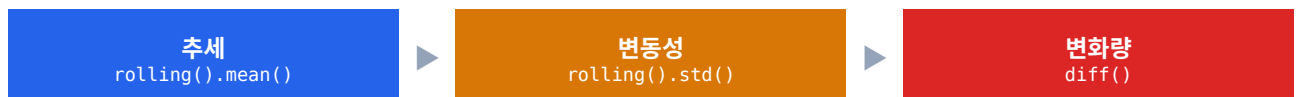
오늘의 한 문장. **"분석보다 전처리가 먼저이고, 전처리의 순서가 결과를 바꾼다."** 22장 전체가 이 한 문장의 전개입니다. 격자를 만들기 전에 채우면 **어디가 비었는지조차 알 수 없습니다**. 정규화 → 결측 확인 → 채우기, 이 순서를 뒤집으면 조용히 틀린 데이터가 만들어집니다.

CHAPTER 0

오늘의 지도

세 관점과 현실 데이터

0-1. 한 센서를 보는 세 관점



관점	물는 질문	비유	잡아내는 것
추세	전체적으로 어디로 가는가	멀리서 본 큰 흐름	서서히 진행되는 열화
변동성	얼마나 불안한가	흔들림의 폭	불안정해지는 구간
변화량	직전 대비 얼마나 움직였나	매 순간의 발걸음	한순간의 급변

개념

변화량은 한 시점 값에서 바로 앞 시점 값을 뺀 차이입니다. 어제 70kg, 오늘 69.5kg이면 변화량은 -0.5kg입니다. 값 자체가 아니라 값의 움직임에 주목하는 관점이고, 고장이 종종 급격한 변화로 나타나기 때문에 유용합니다.

오늘의 출발점 — 어제 두 개에 하나를 더합니다

```
# 세 관점을 한 화면에 (21_02 실습 2)
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, figsize=(11, 8), sharex=True)

ax1.set_title("원본 vs 이동평균")
ax1.plot(engine2["time_cycles"], engine2["s_3"], label="원본", alpha=0.4)
ax1.plot(engine2["time_cycles"], engine2["s_3"].rolling(20).mean(),
label="이동평균")

ax2.set_title("이동표준편차")
ax2.plot(engine2["time_cycles"], engine2["s_3"].rolling(20).std())

ax3.set_title("변화량")
ax3.plot(engine2["time_cycles"], engine2["s_3"].diff())

plt.tight_layout()
plt.show()
```

- ▶ 위 칸은 매끈한 상승, 가운데 칸은 후반에 커지는 흔들림,
- ▶ 아래 칸은 0 근처를 오가다 몇 군데서 크게 튀는 모양

세 칸을 위아래로 놓고 sharex=True로 가로축을 묶으면 같은 회차에서 세 신호가 어떻게 움직이는지 한눈에 대조할 수 있습니다. 21_03에서 배울 "여러 신호가 동시에 평소와 다를 때 이상 의심"의 출발점입니다.

0-2. 학습용 데이터와 현실 데이터

구분		
시간 간격	일정함 (정확히 1시간)	들쭉날쭉 (3~80초)
빈 시점	없음	단절 · 누락
분석 전 처리	거의 불필요	정규화 + 누락 처리 필수
오늘의 파일	21_engine1_timestamp	22_열처리.csv

설비 적용

설비 데이터는 시간 간격이 일정하지 않고 중간이 비어 있습니다. 10초 주기로 설정해도 실제로는 12초 뒤에 들어오거나 80초 동안 통째로 비어 있습니다. 통신 지연·센서 단절·시스템 부하 같은 현장 사정 때문입니다.

문제는 이동평균·표준편차·상관계수가 모두 일정 간격을 전제로 동작한다는 점입니다. "값 6개의 평균"이 어떤 때는 60초, 어떤 때는 480초를 뜻하게 되면 결과 해석 자체가 불가능해집니다.

그래서 22장의 목표는 새 라이브러리가 아니라 이미 아는 **pandas**로 현실 데이터의 시간 축을 바로잡는 것입니다.

0-3. 전처리 파이프라인 6단계

오늘 후반부 전체가 이 여섯 단계입니다. 순서가 정해져 있고, 뒤집으면 안 됩니다.

단계	무엇	도구	오늘 STEP
① 시간 인덱스	변환 · 인덱스 · 정렬	<code>to_datetime</code> <code>set_index</code>	복습 (27일차)
② 간격 진단	측정 간격과 기준 주기 확인	<code>index.diff()</code> <code>value_counts</code>	STEP 5
③ 정규화	일정 격자 생성	<code>asfreq</code> <code>resample</code>	STEP 6
④ 결측 확인	개수 · 비율 · 위치 · 패턴	<code>isna</code> <code>groupby</code>	STEP 6
⑤ 채우기	센서 성격에 맞는 보간	<code>ffill</code> <code>interpolate</code>	STEP 7
⑥ 재확인	남은 NaN과 그래프 품질	<code>isna</code> <code>plot</code>	STEP 7

주의

③ 정규화가 ⑤ 채우기보다 먼저여야 합니다. 정규화하지 않은 원본은 빠진 시각의 행 자체가 없어 보간할 대상조차 식별할 수 없습니다. 격자를 먼저 만들어 "여기 칸이 비었다"를 드러낸 뒤에야 채울 수 있습니다. 순서를 뒤집으면 채우기가 실패하고 간격도 여전히 불규칙합니다.

0-4. 오늘 쓰는 22_열처리.csv 살펴보기

22_01·22_02에서 계속 쓰는 데이터

```
import pandas as pd

df = pd.read_csv("Data/22_열처리.csv")
print(df.shape)
print(df.head(3))
print("원본 결측:", df.isna().sum().to_dict())
```

```
▶ (188, 4)
▶               timestamp  설비  제어출력  소입로온도
▶ 0  2024-03-01 09:00:00    1    0.426    861.05
▶ 1  2024-03-01 09:00:10    1    0.463    860.99
▶ 2  2024-03-01 09:00:20    1    0.438    861.05
▶
▶ 원본 결측: {'설비': 0, '제어출력': 0, '소입로온도': 0}
```

컬럼	종류	값	의미
timestamp	시각	2024-03-01 09:00:00 ~ 09:33:10	측정 시각
설비	범주	1 (한 대)	설비 번호
제어출력	숫자	0.4 근처	제어 출력 비율
소입로온도	숫자	861 근처	소입로 온도(°C)

개념

원본에는 결측이 하나도 없습니다. `isna().sum()`이 전부 0입니다. 그런데 이 데이터의 문제는 **비어 있는 값이 아니라 비어 있는 시각**입니다.

188개 행이 34분 동안 기록되었는데, 10초 간격이면 200개가 있어야 합니다. **행 자체가 없어서 결측으로 잡히지 않는 것**입니다. 이것이 22장이 다루는 문제의 본질이고, STEP 6에서 `asfreq`로 격자에 올리는 순간 **22개의 빈 칸이 드러납니다**.

STEP 1

변화량으로 급변 잡기

diff · pct_change · 해석 주의

643°C 원본에서는 1°C 변화가 묻히지만, 변화량 그래프에서는 그 1°C가 우뚝 솟습니다. 이것이 변화량을 따로 보는 이유입니다.

1-1. 절대 변화량과 변화율

구분		
재는 것	변한 양 그대로	원래 값 대비 비율
100 → 105	+5	+5%
메서드	<code>.diff()</code>	<code>.pct_change()</code>
쓸 때	실제로 얼마나 움직였나 (온도 1°C 상승)	크기 다른 센서를 공정 비교할 때
예	10에서 2 변한 것과 1000에서 2 변한 것이 같은 2	앞은 20%, 뒤는 0.2%로 갈림

메서드 `.diff()` · `.pct_change()` 변한 양과 변한 비율

정의 `.diff()`는 직전 값과의 차, `.pct_change()`는 직전 값 대비 비율을 돌려줍니다. 둘 다 첫 값은 뺄 대상이 없어 NaN입니다.

형태 `series.diff()` · `series.pct_change()`

```
s = pd.Series([100, 103, 102, 108, 107])

print(s.diff().tolist())
print(s.pct_change().round(4).tolist())

▶ [nan, 3.0, -1.0, 6.0, -1.0]
▶ [nan, 0.03, -0.0097, 0.0588, -0.0093]
```

왜 쓰나 첫 값이 NaN인 것은 변화량의 자연스러운 성질이지 오류가 아닙니다. 비교할 앞 값이 없기 때문입니다. 27 일차에서 `.diff().dropna()`를 쓴 이유가 여기 있습니다.

응용 `.pct_change()`는 비율이므로 100을 곱해 퍼센트로 보는 편이 읽기 좋습니다. 그리고 원래 값이 0이면 무한대가 나옵니다.

```
# 퍼센트로 보기
print((s.pct_change() * 100).round(2).tolist())
```

```
# 0으로 나누면 inf
z = pd.Series([0, 5, 10])
print(z.pct_change().tolist())
```

```
▶ [nan, 3.0, -0.97, 5.88, -0.93]
▶ [nan, inf, 1.0]
```

개념

.pct_change()의 결과가 inf나 -inf면 **원래 값에 0이 섞여 있다**는 뜻입니다. 설비 데이터에서 정지 구간의 값이 0이면 재가동 시점에 무한대가 나옵니다. np.isinf()로 확인하거나 .replace([np.inf, -np.inf], np.nan)으로 처리해야 합니다.

주의

.diff(periods=n)으로 **n칸 앞**과 비교할 수 있습니다. pct_change도 마찬가지입니다. 하루 전이 아니라 일주일 전과 비교하고 싶을 때 유용합니다.

쓰임	코드	언제
직전 대비 변화	s.diff()	급변 시점 찾기
n칸 앞과 비교	s.diff(periods=7)	주 단위 변화를 볼 때
변화 비율	s.pct_change() * 100	크기 다른 센서 비교
누적 변화	s - s.iloc[0]	시작 대비 총 변화
평활 후 변화	s.rolling(5).mean().diff()	노이즈를 걸러낸 급변

1-2. 실습 1 — 변화량으로 급변 탐지

Practice/21_02_example.ipynb — 실습 1

```
df = pd.read_csv("Data/21_cmapss_fd001_sample.csv")
engine2 = df[df["unit_nr"] == 2]
print(engine2.shape)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 6), sharex=True)

ax1.set_title("원래 데이터")
ax1.plot(engine2["time_cycles"], engine2["s_11"])

ax2.set_title("변화량 데이터")
diff = engine2["s_11"].diff()
ax2.plot(engine2["time_cycles"], diff)

# 0 기준선 긋기
ax2.axhline(0, color="yellow")

plt.tight_layout()
plt.show()

print("최대 급변량:", round(diff.abs().max(), 2))
```

▶ (287, 10)
▶ 최대 급변량: 0.54

개념

변화량 그래프는 평소에 0 근처에 머물다가 급변 시점에만 크게 튕니다. 원본에서는 47~51 사이를 오르내리는 선이라 어디가 급변인지 눈에 안 들어오지만, 변화량으로 바꾸면 **0을 기준으로 위아래로 튀는 막대 모양**이 되어 즉시 보입니다.

ax2.axhline(0)으로 **0 기준선을 긋는 것이** 중요합니다. 변화량 그래프에서 0은 "변화 없음"이라는 기준점이므로, 선이 없으면 어디가 위이고 아래인지 판단하기 어렵습니다.

급변 시점이 센서마다 다릅니다

```
# 세 센서의 급변량 비교 — 어느 센서가 가장 요동치나
for s in ["s_11", "s_3", "s_4"]:
    d = engine2[s].diff()
    회차 = engine2["time_cycles"].iloc[d.abs().argmax()]
    print(f"{s:5} 최대 급변 {d.abs().max():5.2f}"
          f" | 변화량 std {d.std():.4f} | 발생 회차 {회차}")
```

```
▶ s_11 최대 급변 0.54 | 변화량 std 0.1837 | 발생 회차 177
▶ s_3  최대 급변 4.73 | 변화량 std 1.4261 | 발생 회차 236
▶ s_4  최대 급변 3.29 | 변화량 std 1.0771 | 발생 회차 268
```

설비 적용

세 센서의 최대 급변 시점이 177 · 236 · 268회차로 제각각입니다. 만약 세 센서가 같은 회차에서 동시에 튀었다면 그것은 설비 전체에 영향을 준 사건일 가능성이 큼니다. 흩어져 있다는 것은 각각 우연한 잡음일 가능성이 높다는 뜻입니다. 여러 센서를 함께 봐야 판단할 수 있다는 것이 STEP 3~4의 주제입니다.

1-3. 급변 시점 해석 시 주의

큰 변화량은 주목 신호이지 고장 확정이 아닙니다. 원인이 세 가지나 됩니다.

원인	무슨 일	확인 방법
① 노이즈	우연히 한 시점이 크게 튀면 변화량도 따라 튼	이동평균 후 변화량으로 평활
② 운영 변화	가동·정지, 부하 변경 시 정상적으로 크게 변함	운영 기록과 교차 확인
③ 측정 오류	센서 오작동이나 결측 복구 시점에서 값이 튼	누락 점검 습관화

이동평균 → 변화량 순서로 계산

```
# 노이즈를 걸러낸 뒤 변화량 보기
raw_diff = engine2["s_3"].diff()
ma_diff = engine2["s_3"].rolling(5).mean().diff()

print("원본 변화량 std :", round(raw_diff.std(), 4))
print("평활 후 변화량 std:", round(ma_diff.std(), 4))
```

```
▶ 원본 변화량 std : 1.4261
▶ 평활 후 변화량 std: 0.2770
```

주의

변화량은 노이즈에 매우 민감합니다. 원본을 그대로 `diff()`하면 잡음이 그대로 증폭됩니다. 위 결과처럼 이동평균을 먼저 걸고 변화량을 구하면 흔들림이 5분의 1로 줄어 진짜 급변만 남습니다. 다만 이동평균 때문에 급변 시점이 뒤로 밀리는 지연(lag)이 생기므로, 시점의 정확성이 중요하다면 원본 변화량을 함께 봐야 합니다.

결국 원본 값, 다른 센서, 운영 기록을 함께 보고 종합 판단해야 합니다. 변화량 그래프 하나만 보고 "여기가 고장"이라고 결론 내리면 세 원인 중 어느 쪽인지 구별할 수 없습니다.

1-4. 급변 후보를 숫자로 세기

그래프에서 눈으로 짚는 대신 기준을 정해 자동으로 세면 센서끼리 비교할 수 있습니다.

센서마다 임계값이 다릅니다

```
# 변화량이 자기 표준편차의 2배를 넘으면 급변 후보
for s in ["s_2", "s_3", "s_4", "s_7", "s_11"]:
    d = engine2[s].diff()
    임계 = d.std() * 2
    후보 = engine2["time_cycles"][d.abs() > 임계]
    print(f"{s:5} 임계 {임계:6.3f} | 급변 후보 {len(후보):2}회차"
          f" | 앞 5개 {후보.tolist()[:5]}")
```

- ▶ s_2 임계 1.296 | 급변 후보 17회차 | 앞 5개 [8, 13, 18, 34, 39]
- ▶ s_3 임계 2.852 | 급변 후보 17회차 | 앞 5개 [141, 159, 186, 202, 208]
- ▶ s_4 임계 2.154 | 급변 후보 14회차 | 앞 5개 [3, 4, 13, 18, 40]
- ▶ s_7 임계 0.883 | 급변 후보 9회차 | 앞 5개 [106, 117, 118, 119, 169]
- ▶ s_11 임계 0.367 | 급변 후보 12회차 | 앞 5개 [23, 62, 68, 151, 169]

설비 적용

임계값을 센서마다 따로 계산하는 것이 요령입니다. s_2는 1.296, s_3은 2.852로 세 배 가까이 차이 납니다. 하나의 절대 기준을 쓰면 어떤 센서는 전부 걸리고 어떤 센서는 하나도 안 걸립니다.

그리고 후보 시점의 분포가 다릅니다. s_2와 s_4는 앞쪽(3~40회차)에 몰려 있고 s_3은 뒤쪽(141회차 이후)에 몰려 있습니다. s_7은 117·118·119회차가 연속으로 걸리는데, 이런 연속 급변은 우연보다 실제 사건일 가능성이 높습니다.

s_7의 106·117·118·119와 s_11의 169가 겹치지 않는다는 점도 정보입니다. 여러 센서가 같은 회차에서 동시에 걸릴 때 비로소 설비 전체의 사건으로 볼 수 있습니다.

STEP 2

주기성과 시계열 분해

주기 개념 · resample · decompose

추세가 **한 방향**이라면 주기성은 **같은 모양의 되풀이**입니다. 그리고 이 정상 주기가 흐트러지면 그것이 이상 신호입니다.

2-1. 주기성이란

주기	간격	예	설비에서
하루 주기	24시간	기온 · 전력 · 교통량	낮 가동 · 밤 정지
일주일 주기	7일	평일과 주말의 차이	근무 · 휴일 리듬
설비 고유 주기	설비마다 다름	가동 · 정지 간격	가열 · 냉각 반복

개념

주기 길이는 같은 패턴이 한 번 반복되는 데 걸리는 간격입니다. 그래프에서 한 봉우리에서 다음 봉우리까지의 거리로 가늠합니다.

입문 단계에서는 **눈으로 재는 것으로 충분**합니다. 복잡한 수학보다 "이 데이터에 주기가 있는가, 대략 어느 간격인가"를 파악하는 감각이 먼저입니다.

2-2. 실습 3~4 — 시간 축으로 보기

Practice/21_02_example.ipynb — 실습 3

```
ts = pd.read_csv("Data/21_engine1_timestamp_sample.csv")
print("shape:", ts.shape)

ts["timestamp"] = pd.to_datetime(ts["timestamp"])
ts = ts.set_index("timestamp").sort_index()
print("기간:", ts.index[0], "~", ts.index[-1])
print("결측:", ts.isna().sum().sum())

# 빠진 시각이 있는지도 확인
전체시각 = pd.date_range(ts.index[0], ts.index[-1], freq="1h")
print("이론상", len(전체시각), "| 실제", len(ts), "| 빠짐", len(전체시각) - len(ts))
```

▶ shape: (192, 6)
▶ 기간: 2024-01-01 00:00:00 ~ 2024-01-08 23:00:00
▶ 결측: 0
▶ 이론상 192 | 실제 192 | 빠짐 0

연결

어제 쓴 20_engine01_timestamp_sample.csv는 10개 시각이 빠져 있었지만, 오늘의 21_engine1_timestamp_sample.csv는 192행이 전부 채워져 있습니다. 이름이 비슷해도 다른 파일입니다. 오늘 것이 완전하기 때문에 STEP 2에서는 결측 걱정 없이 주기와 분해에 집중할 수 있습니다.

Practice/21_02_example.ipynb — 실습 4

```
# 원본 · 6시간 · 하루 평균을 겹쳐 그리기
h6 = ts["s_4"].resample("6h").mean()
day = ts["s_4"].resample("D").mean()

plt.plot(ts.index, ts["s_4"], alpha=0.3, label="raw(1h)")
plt.plot(h6.index, h6, marker="o", label="6h mean")
plt.plot(day.index, day, marker="s", label="daily mean")
plt.legend()
plt.show()

print("6시간 개수:", len(h6), "| 하루 개수:", len(day))
print(day.round(2).tolist())
```

▶ 6시간 개수: 32 | 하루 개수: 8
▶ [1401.21, 1403.43, 1405.43, 1407.8, 1409.91, 1412.0, 1414.04, 1416.37]

개념

192개가 8개로 줄었습니다. 그리고 그 8개는 1401.21에서 1416.37까지 **한 번도 안 꺾이고 오릅니다.** 리샘플링이 노이즈를 상쇄하고 큰 패턴을 드러낸 것입니다.

marker="o"와 marker="s"로 **샘플링 시점에 점을 찍는 것이** 요령입니다. 선만 그리면 6시간 집계와 하루 집계의 해상도 차이가 안 보이지만, 점을 찍으면 **얼마나 거칠어졌는지** 눈에 들어옵니다.

주의

같은 데이터도 묶는 단위에 따라 보이는 패턴이 달라집니다. 너무 크게 묶으면 짧은 주기가 사라지고, 너무 작으면 노이즈가 남습니다. 원칙은 **보고 싶은 패턴보다 작은 단위로 묶어야 그 패턴이 살아남는다는 것**입니다. 하루 주기를 보고 싶으면 하루보다 작은 단위(6시간 등)로 묶어야 합니다. 그리고 **결과를 보고할 때는 어떤 단위로 봤는지 함께 밝혀야** 합니다.

2-3. 실습 5 — ffill과 보간 비교

Practice/21_02_example.ipynb — 실습 5

```
import numpy as np
```

```
# 작은 예시로 차이 확인
```

```
s = pd.Series([10.0, np.nan, np.nan, 16.0, 18.0])
print("ffill      :", s.fffll().tolist())
print("interpolate:", s.interpolate().tolist())
```

```
# 실데이터 일부를 일부러 비우고 복원
```

```
s2 = ts["s_4"].copy()
s2.iloc[50:55] = np.nan
print("누락 개수:", int(s2.isna().sum()))
print("보간 후 :", int(s2.interpolate().isna().sum()))
```

```
▶ ffill      : [10.0, 10.0, 10.0, 16.0, 18.0]
```

```
▶ interpolate: [10.0, 12.0, 14.0, 16.0, 18.0]
```

```
▶ 누락 개수: 5
```

```
▶ 보간 후 : 0
```

개념

ffill은 [10, 10, 10, 16, 18]로 계단이 되고, interpolate는 [10, 12, 14, 16, 18]로 매끄럽게 이어집니다. 10과 16 사이를 직선으로 잇고 그 위의 값을 가져오기 때문입니다. 온도처럼 **서서히 변하는 값에는 보간이 훨씬 자연스럽고**, 설비 모드나 설정값처럼 **유지되는 값에는 ffill이 맞습니다.** STEP 7에서 이 선택 기준을 자세히 다룹니다.

2-4. 실습 6 — 시계열 분해

함수 `seasonal_decompose()` 시계열을 추세·계절·잔차로 분리

정의 statsmodels의 함수로, 시계열을 **추세(trend) + 계절(seasonal) + 잔차(resid)** 세 성분으로 자동 분리합니다.

형태 `seasonal_decompose(series, model="additive", period=24)`

```
from statsmodels.tsa.seasonal import seasonal_decompose

# period=24 : 1시간 간격에서 하루 주기가 반복된다고 알려주는 값
# model="additive" : 원본 = 추세 + 계절 + 잔차 로 본다는 뜻
result = seasonal_decompose(ts["s_4"], model="additive", period=24)

fig = result.plot()
fig.set_size_inches(11, 8)
plt.tight_layout()
plt.show()
```

▶ 원본 · 추세 · 계절 · 잔차 네 칸이 위아래로 그려집니다

왜 쓰나 노이즈와 반복 주기를 분리하고 순수한 장기 신호만 남기기 위한 전처리 기법입니다. 열화 속도만 보고 싶을 때 계절 성분을 건어낼 수 있습니다.

응용 그림만 보지 말고 **성분별 숫자를 확인**해야 이 분해가 의미 있는지 판단할 수 있습니다.

```
for 이름, 성분 in [("trend", result.trend),
                  ("seasonal", result.seasonal),
                  ("resid", result.resid)]:
    print(f"{이름:9} NaN {성분.isna().sum():3}개"
          f" | 범위 {성분.min():9.3f} ~ {성분.max():9.3f}"
          f" | 표준편차 {성분.std():7.3f}")

print("원본 표준편차:", round(ts["s_4"].std(), 3))
```

▶ trend NaN 24개 | 범위 1401.277 ~ 1416.340 | 표준편차 4.370
▶ seasonal NaN 0개 | 범위 -0.453 ~ 0.338 | 표준편차 0.189
▶ resid NaN 24개 | 범위 -2.105 ~ 2.612 | 표준편차 0.713
▶ 원본 표준편차: 5.044

개념

seasonal_decompose는 **결측이 있으면 오류를 냅니다**. 반드시 interpolate() 등으로 결측을 완전히 없앤 뒤 호출해야 합니다. 그리고 period는 **우리가 가정으로 넣는 값**입니다. 데이터가 알아서 주기를 찾아 주는 것이 아닙니다.

주의

model="additive"는 **원본 = 추세 + 계절 + 잔차**로 봅니다. 계절 진폭이 값 크기에 비례해 커지는 데이터라면 model="multiplicative"(곱하기 모형)가 맞습니다. 설비 센서는 보통 additive로 충분합니다.

2-5. 분해 결과를 검증하기

additive 모형이 맞는지 직접 확인

```
# 세 성분을 더하면 원본이 나와야 합니다
합 = result.trend + result.seasonal + result.resid
비교 = pd.DataFrame({"원본": ts["s_4"], "성분합": 합}).dropna()
print("추세 + 계절 + 잔차 == 원본 인가?:", np.allclose(비교["원본"], 비교["성분합"]))

# 추세 앞뒤가 비어 있는 이유
print("추세 첫 유효 시각:", result.trend.first_valid_index())
print("추세 끝 유효 시각:", result.trend.last_valid_index())
print("유효 구간:", result.trend.notna().sum(), "/", len(result.trend))
```

- ▶ 추세 + 계절 + 잔차 == 원본 인가?: True
- ▶ 추세 첫 유효 시각: 2024-01-01 12:00:00
- ▶ 추세 끝 유효 시각: 2024-01-08 11:00:00
- ▶ 유효 구간: 168 / 192

개념

추세의 앞 12칸과 뒤 12칸이 비어 있습니다. 추세 성분이 period=24의 **중심 이동평균**이기 때문입니다. 어제 배운 rolling(20)이 앞 19칸을 비웠던 것과 같은 이유인데, center=True라 **앞뒤로 나눠서** 비는 점이 다릅니다. $192 - 24 = 168$ 이 유효 구간입니다.

분해 결과를 숫자로 검산

```
# 분해 추세가 정말 중심 이동평균과 같은지 확인
ma24 = ts["s_4"].rolling(window=24, center=True).mean()
print("두 선의 상관계수:", round(result.trend.corr(ma24), 4))

# 계절 성분이 실제로 하루 주기 무늬인지
계절 = result.seasonal.groupby(result.seasonal.index.hour).first()
print("계절 성분 진폭:", round(계절.max() - 계절.min(), 3))
print("원본 대비 비중:", round(result.seasonal.std() / ts["s_4"].std() * 100, 1),
"%")
```

- ▶ 두 선의 상관계수: 1.0
- ▶ 계절 성분 진폭: 0.791
- ▶ 원본 대비 비중: 3.7 %

설비 적용

분해 **trend**와 **rolling(24, center=True)**의 상관이 정확히 1.0입니다. 둘은 사실상 같은 계산입니다. 어제 배운 이동평균을 이미 알고 있으면 분해의 추세 성분도 이미 아는 셈입니다.

그리고 계절 성분의 진폭이 0.791로 원본 대비 3.7%밖에 안 됩니다. 잔차의 표준편차 0.713이 계절의 0.189보다 훨씬 큼니다. 즉 이 데이터에는 하루 주기 무늬가 사실상 없습니다. **period=24**는 우리가 가정으로 넣은 값일 뿐입니다.

교안도 "C-MAPSS는 뚜렷한 주기가 약해 분해 효과가 크진 않지만, 도구 작동 방식을 알아두면 좋다"고 명시합니다. 분해를 돌렸는데 계절 성분이 작다면 "주기가 없다"는 결론이지 실패가 아닙니다.

STEP 3

센서 간 관계 보기

corr · heatmap · 다중공선성

한 센서만 보는 것은 체온 하나만 보고 진단하는 것과 같습니다. 설비는 부품이 맞물린 시스템이라, 여러 센서에 흔적이 남는 것이 진짜 고장입니다.

3-1. 센서 하나로 부족한 이유

관점	내용	판정 기준
① 한계	한 센서만으로는 오판과 놓침의 위험	—
② 구조	설비는 부품이 맞물린 시스템	센서는 서로 다른 부분을 측정
③ 판별	함께 변하면 진짜, 하나만 튀면 센서 문제	여러 센서에 남는 흔적

개념

"함께 본다"는 것은 그래프를 여러 개 그린다는 뜻이 아니라 센서들이 어떻게 같이 움직이는지 보는 일입니다.

부품이 연결되어 있으면 센서 사이에 일정한 관계가 유지되고, 그 평소 관계가 깨지는 순간이 이상 신호입니다. 늘 함께 다니던 두 사람이 따로 다니기 시작하면 무슨 일이 있었나 싶은 것과 같습니다.

3-2. 실습 1 — 상관 heatmap

Practice/21_03_example.ipynb — 실습 1

```
df = pd.read_csv("Data/21_cmapss_fd001_sample.csv")
sensors = ["s_2", "s_3", "s_4", "s_7", "s_11"]

e2 = df[df["unit_nr"] == 2].reset_index(drop=True)

sns.heatmap(e2[sensors].corr(), annot=True, cmap="coolwarm", vmin=-1, vmax=1)
plt.title("Engine 2 sensor correlation")
plt.show()

print(e2[sensors].corr().round(3))
```

	s_2	s_3	s_4	s_7	s_11
s_2	1.000	0.954	0.986	-0.932	0.810
s_3	0.954	1.000	0.975	-0.957	0.827
s_4	0.986	0.975	1.000	-0.953	0.823
s_7	-0.932	-0.957	-0.953	1.000	-0.815
s_11	0.810	0.827	0.823	-0.815	1.000

키워드 vmin= · vmax= 색 눈금의 범위를 고정

정의 heatmap의 색이 어느 값에서 가장 진해질지를 직접 지정합니다. 상관계수는 -1~1이므로 그 범위로 고정합니다.

형태 sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1)

```
# x vmin/vmax 없이 — 데이터 최솟값이 가장 진한 파랑이 됨
sns.heatmap(e2[sensors].corr(), annot=True, cmap="coolwarm")

# ✓ 범위를 고정 — 다른 엔진과 비교해도 색의 의미가 같음
sns.heatmap(e2[sensors].corr(), annot=True, cmap="coolwarm",
            vmin=-1, vmax=1)
```

▶ 범위를 고정하면 0이 흰색, +1이 진한 빨강, -1이 진한 파랑으로 항상 같습니다

왜 쓰나 여러 엔진의 heatmap을 비교할 때 필수입니다. 고정하지 않으면 엔진마다 색 기준이 달라져 같은 진한 파랑이 -0.9일 수도 -0.3일 수도 있습니다.

응용 26일차에 배운 center=0과 목적이 비슷하지만 방식이 다릅니다. center는 **중앙만** 고정하고 양 끝은 데이터에 맞추며, vmin·vmax는 **양 끝을 직접** 고정합니다.

```
# 두 방식 비교
sns.heatmap(corr, cmap="coolwarm", center=0)          # 중앙만 고정
sns.heatmap(corr, cmap="coolwarm", vmin=-1, vmax=1)    # 양 끝 고정

# 상관행렬에는 vmin/vmax가 더 정확합니다
```

개념

상관계수는 **이론적 범위가 -1~1로 정해져 있으므로** `vmin=-1, vmax=1`이 가장 정직한 선택입니다. 반면 온도나 압력처럼 범위가 정해지지 않은 값의 heatmap이라면 `center`만 쓰는 편이 낫습니다.

주의

`annot=True`와 `fmt=".2f"`는 26일차에 배운 그대로입니다. 다만 실습 코드는 `fmt`를 생략했는데, 그러면 0.98599처럼 길게 나올 수 있습니다. `fmt=".2f"`를 함께 쓰는 습관을 들이십시오.

3-3. 실습 2 — 강하게 연동된 묶음 찾기

Practice/21_03_example.ipynb — 실습 2

```
corr = e2[sensors].corr()
```

```
# 대각선을 빼고 절댓값 0.9 이상인 쌍만
for i in range(len(sensors)):
    for j in range(i + 1, len(sensors)):
        c = corr.iloc[i, j]
        if abs(c) >= 0.9:
            print(sensors[i], "-", sensors[j], ":", round(c, 2))
```

```
▶ s_2 - s_3 : 0.95
▶ s_2 - s_4 : 0.99
▶ s_2 - s_7 : -0.93
▶ s_3 - s_4 : 0.97
▶ s_3 - s_7 : -0.96
▶ s_4 - s_7 : -0.95
```

개념

`for i in range(len)` 안에 `for j in range(i + 1, len)`를 넣는 것이 **중복 없이 모든 쌍을 도는 관용구**입니다. `j`가 `i + 1`부터 시작하므로 (`s_2, s_3`)은 보지만 (`s_3, s_2`)는 안 봅니다. 상관행렬이 대칭이라 절반만 보면 충분하기 때문입니다. 26일차에 `unstack()`으로 하던 일을 반복문으로 하는 방식입니다.

묶음	센서	상관계수	해석
양의 묶음	$s_2 \cdot s_3 \cdot s_4$	+0.95 ~ +0.99	함께 오르는 세 센서
음의 관계	s_7 대 위 세 개	-0.93 ~ -0.96	반대로 내려가는 센서
약한 관계	s_{11}	0.81 ~ 0.83	0.9 미만이라 목록에서 빠짐

설비 적용

네 센서가 사실상 같은 현상을 다른 부호로 재고 있습니다. $s_2 \cdot s_3 \cdot s_4$ 가 서로 0.95 이상이고 s_7 이 이들과 -0.93 이하라면, 네 값 중 하나만 알아도 나머지 셋을 거의 맞힐 수 있습니다.

통계에서는 이것을 다중공선성(multicollinearity)이라고 합니다. 나중에 예측 모델을 만들 때 네 센서를 다 넣어도 정보가 늘지 않고 오히려 모델이 불안정해집니다. 대표 하나만 남기거나 합쳐서 쓰는 것이 보통입니다.

s_{11} 은 0.81~0.83으로 혼자 조금 다르게 움직입니다. 다중공선성 관점에서는 오히려 s_{11} 이 추가 정보를 가진 센서입니다. 어제 s_{11} 의 변동성만 +64.9%로 크게 늘었던 것과 같은 맥락으로 읽을 수 있습니다.

3-4. 상관은 인과가 아닙니다

개념

아이스크림 판매량과 물놀이 사고는 함께 늘어나지만, 아이스크림이 사고를 일으키는 것이 아닙니다. 둘 다 "여름"이라는 공통 원인 때문입니다.

오늘 센서들이 다 연관된 것도 같은 구조입니다. s_2 가 s_4 를 변하게 만든 것이 아니라 둘 다 "엔진 열화"라는 공통 원인 때문에 함께 움직이는 것입니다. A 때문에 B가 변했다고 단정하면 안 됩니다.

STEP 4

변화점과 이상 구간

axvline · axvspan · 정상 범위

전체 한 값이 사진이라면 이동통계는 동영상입니다. 고장이 진행되는 과정을 추적하려면 동영상의 관점이 필요합니다.

4-1. 변화점이란

구분	내용	예
정의	데이터 성질이 그 전후로 뚜렷이 달라지는 시점	피로를 느끼기 시작한 어느 날
형태	추세 변화 · 변동성 증가 · 수준 이동	기울기가 꺾이거나 흔들림이 커짐
의미	이상이 시작된 시점	이동통계 그래프에서 눈으로 짚기

개념

입문 단계에서는 **변화점을 자동으로 찾는 알고리즘까지 갈 필요 없습니다**. 이동평균·이동표준편차 그래프를 보고 **직접 짚는 것으로 충분합니다**. 중요한 것은 "여기서부터 달라졌다"를 근거를 갖고 말할 수 있느냐입니다.

4-2. 이상 의심 구간 판단 기준

한 신호만으로는 약하고, 여러 신호가 같은 구간에서 동시에 평소와 다를 때 근거가 강해집니다.

신호	확인 도구	오늘 데이터 (2번 엔진 s_3)
① 추세 변화	<code>rolling().mean()</code>	$r=+0.988$ 로 계속 상승
② 변동성 증가	<code>rolling().std()</code>	후반 +53.3% (27일차 확인)
③ 급변	<code>diff()</code>	236회차에서 최대 4.73
④ 관계 붕괴	초반·후반 상관 비교	초반 0.9 → 후반 0.3 같은 변화

설비 적용

네 가지 신호가 같은 구간에 겹치면 확신이 매우 강해집니다. 반대로 신호 하나만 나타나면 일시적 외란 가능성이 있으므로 이상으로 단정하지 말고 관찰 대상으로 유지합니다. 그리고 판단 기준은 **절대 기준이 아니라 그 설비의 평소와 비교**하는 것입니다. 같은 진동값도 설비마다 정상 범위가 다릅니다.

4-3. 실습 3 — 의심 구간 색칠하기

함수 `plt.axvspan()` · `plt.axvline()` 구간을 색칠하고 시점에 선 긋기

정의 `axvline`은 세로선 하나, `axvspan`은 가로 방향 구간을 색으로 채웁니다. 어제 배운 `axhline`의 세로 버전입니다.

형태 `plt.axvline(x=187, color="red", linestyle="--")` · `plt.axvspan(180, 192, color="red", alpha=0.15)`

```
std20 = e2["s_3"].rolling(20).std()

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(11, 6), sharex=True)

ax1.plot(e2["time_cycles"], e2["s_3"])
ax1.axvspan(265, 287, color="red", alpha=0.15)
ax1.set_title("s_3 raw with suspect zone")

ax2.plot(e2["time_cycles"], std20, color="red")
ax2.axvspan(265, 287, color="red", alpha=0.15)
ax2.set_title("s_3 moving std(20)")

plt.tight_layout()
plt.show()
```

▶ 위아래 두 칸에 같은 위치의 붉은 띠가 그려져 구간이 대조됩니다

왜 쓰나 "265회차 이후"라고 말로만 하는 것보다 색으로 칠하면 즉시 전달됩니다. 그리고 위아래 칸에 같은 구간을 칠하면 원본과 변동성이 같은 구간에서 달라졌다는 것이 한눈에 보입니다.

응용 `alpha`를 작게 해야 아래 데이터가 비쳐 보입니다. 너무 많이 칠하면 핵심이 묻힙니다.

```
# 변화점 하나만 표시할 때는 세로선
plt.axvline(x=265, color="red", linestyle="--", label="변화점")

# 여러 구간을 칠할 때는 반복문
for 시작, 끝 in [(120, 135), (265, 287)]:
    plt.axvspan(시작, 끝, color="orange", alpha=0.12)
```

개념

`alpha=0.15` 정도가 적당합니다. 0.3을 넘으면 아래 선이 잘 안 보이고, 0.05 이하면 칠한 것이 눈에 안 띕니다. `axvspan`은 그래프 범례에 잘 안 잡히므로, 무엇을 칠한 것인지 제목이나 주석으로 설명해야 합니다.

주의

sharex=True로 두 칸의 가로축을 묶으면 **한쪽을 확대할 때 다른 쪽도 함께 확대**됩니다. 위아래로 비교하는 그래프에서는 거의 항상 커는 것이 좋습니다.

4-4. 실습 4 — 정상 범위 벗어난 점

Practice/21_03_example.ipynb — 실습 4

```
s = e2["s_3"]
m, sd = s.mean(), s.std()
lower, upper = m - 2 * sd, m + 2 * sd

print("정상 범위:", round(lower, 2), "~", round(upper, 2))

out = ((s > upper) | (s < lower)).sum()
print("벗어난 점:", int(out))
print("초반 50회차 평균:", round(s.iloc[:50].mean(), 2))
```

- ▶ 정상 범위: 1583.17 ~ 1608.94
- ▶ 벗어난 점: 2
- ▶ 초반 50회차 평균: 1586.96

자주 하는 실수

어제 배운 $\pm 2\sigma$ 밴드와 결과가 완전히 다릅니다. 어제는 268회차 중 17개(6.3%)가 밖이었는데 오늘은 287회차 중 2개뿐입니다. 코드가 틀린 것이 아니라 **기준이 다르기** 때문입니다.

어제는 `rolling(20)`으로 구간마다 움직이는 기준을 만들었고, 오늘은 `s.mean()`과 `s.std()`로 전체 하나의 고정 기준을 만들었습니다. 이 데이터는 값이 1585에서 1606까지 꾸준히 오르기 때문에, **전체 평균 기준으로 보면 대부분이 범위 안에 들어옵니다.**

추세가 있는 데이터에 고정 기준을 쓰면 이상 탐지가 거의 작동하지 않습니다. 오늘 실습 4는 방법을 익히는 예제이고, 실제로는 어제의 이동 밴드가 시계열에 훨씬 적합합니다.

방식	기준	오늘 결과	추세 있는 데이터에서
고정 밴드	전체 $\text{mean} \pm 2 * \text{std}$	2개 이탈	거의 안 잡힘
이동 밴드	<code>rolling(20)</code> 기준	17개 이탈 (27일차)	적합

4-5. 실습 5 — 의심 구간을 자동으로 찾기

실습 3에서는 265~287을 눈으로 보고 손으로 적었습니다. 이것을 기준선을 넘은 구간으로 자동 검출하는 방법입니다.

Practice/21_03_example_5.ipynb — 실습 5

```
ma = e2["s_3"].rolling(window=20).mean()
sd = e2["s_3"].rolling(window=20).std()

# 기준선 = 이동표준편차의 평균 + 표준편차
기준 = sd.mean() + sd.std()
초과 = (sd > 기준).fillna(False)
print("변동성 기준선:", round(기준, 4), "| 기준 초과 회차:", 초과.sum(), "개")

# 연속된 덩어리로 묶기 — 값이 바뀌는 지점마다 번호를 새로 매김
덩어리 = (초과 != 초과.shift()).cumsum()

구간들 = []
for g in 덩어리[초과].unique():
    mask = 초과 & (덩어리 == g)
    구간들.append((e2.loc[mask, "time_cycles"].min(),
                  e2.loc[mask, "time_cycles"].max(),
                  mask.sum()))

for a, b, n in 구간들:
    print(f" {a:3} ~ {b:3} 회차 ({n}개)")
```

▶ 변동성 기준선: 1.3247 | 기준 초과 회차: 47 개

- ▶ 171 ~ 173 회차 (3개)
- ▶ 201 ~ 212 회차 (12개)
- ▶ 217 ~ 217 회차 (1개)
- ▶ 237 ~ 239 회차 (3개)
- ▶ 251 ~ 254 회차 (4개)
- ▶ 261 ~ 263 회차 (3개)
- ▶ 267 ~ 287 회차 (21개)

패턴 `(s != s.shift()).cumsum()` 연속된 덩어리에 번호 매기기

정의 값이 바뀌는 지점마다 1씩 늘어나는 번호를 만듭니다. 같은 값이 이어지는 구간은 같은 번호를 갖게 됩니다.

형태 덩어리 = (조건 != 조건.shift()).cumsum()

```
# 작은 예로 원리 확인
x = pd.Series([False, True, True, False, True, True, True])
print("shift 비교:", (x != x.shift()).tolist())
print("cumsum   :", (x != x.shift()).cumsum().tolist())
```

```
▶ shift 비교: [True, True, False, True, True, False, False]
▶ cumsum   : [1, 2, 2, 3, 4, 4, 4]
```

왜 쓰나 연속 구간의 시작과 끝, 길이를 구하는 표준 관용구입니다. 이상 구간을 찾을 때도, STEP 7에서 연속 결측 길이를 잴 때도 똑같이 씁니다.

응용 번호를 매긴 뒤 groupby로 묶으면 구간별 길이나 통계를 한 번에 구할 수 있습니다.

```
# 구간별 길이를 한 줄로
덩어리 = (초과 != 초과.shift()).cumsum()
print(초과.groupby(덩어리).sum().pipe(lambda s: s[s > 0]).tolist())
```

```
# 가장 긴 구간 고르기
시작, 끝, 길이 = max(구간들, key=lambda r: r[2])
print("가장 긴 구간:", 시작, "~", 끝, "회차 (", 길이, "개)")
```

```
▶ [3, 12, 1, 3, 4, 3, 21]
▶ 가장 긴 구간: 267 ~ 287 회차 ( 21 개 )
```

개념

.shift()는 한 칸 아래로 미는 것이라 첫 값은 항상 NaN과 비교됩니다. False != NaN은 True가 되므로 첫 원소는 항상 새 덩어리로 시작하는데, 이것이 오히려 원하는 동작입니다.

설비 적용

자동 검출이 손으로 짚은 구간을 거의 그대로 찾아냈습니다. 실습 3에서 눈으로 적은 265~287과 자동 검출의 가장 긴 구간 267~287회차(21개)가 사실상 같습니다. 눈이 틀리지 않았다는 것을 숫자로 확인한 셈입니다. 다만 min ~ max로 잡으면 171~287이 되어 너무 넓어지므로, 반드시 연속 덩어리로 나눠야 합니다.

STEP 5

측정 주기 진단

간격 diff · value_counts · 불규칙의 위험

여기서 데이터가 바뀝니다. 22_열처리.csv는 10초 주기로 설정됐는데 실제 간격이 3초부터 80초까지 제각각입니다. 현실 데이터의 모습입니다.

5-1. 측정 주기란

구분	내용	설비에서
정의	센서가 값을 한 번 기록하고 다음 값까지의 시간 간격	영어로 sampling interval
짧으면	짧은 변화까지 포착, 데이터 양 폭증	1초 주기면 1시간에 3600개
길면	가벼운 데이터, 짧은 이상 신호 누락	그 사이의 스파이크를 놓침
고장 예측용	전조가 짧은 순간에 나타나 짧은 주기 필요	10초가 흔한 현실적 설정

개념

이 간격이 일정해야 시계열 분석이 제대로 동작합니다. "값 6개의 평균"이라는 계산이 의미를 가지려면 그 6개가 항상 같은 시간 폭을 덮어야 합니다. 이동평균·이동표준편차·자기상관 — 모두 일정 간격을 전제로 동작합니다.

5-2. 실습 1 — 1분 리샘플링 복습

Practice/22_01_example.ipynb — 실습 1

```
df = pd.read_csv("Data/22_열처리.csv")
print(df.shape)

df["timestamp"] = pd.to_datetime(df["timestamp"])
df = df.set_index("timestamp").sort_index()

one_min = df[["제어출력", "소입로온도"]].resample("1min").mean()
print(one_min.head(3).round(2))
print("1분 평균 행:", len(one_min))
print("1분 평균 빈 칸:", int(one_min["제어출력"].isna().sum()))
```

```
> (188, 4)
>           제어출력  소입로온도
> timestamp
> 2024-03-01 09:00:00  0.44  861.06
> 2024-03-01 09:01:00  0.42  861.18
> 2024-03-01 09:02:00  0.40  861.31
>
> 1분 평균 행: 34
> 1분 평균 빈 칸: 1
```

개념

34분 동안 188개가 기록됐는데 1분 평균을 내니 34칸 중 1칸이 비었습니다. 그 1분 동안 값이 하나도 안 들어온 것입니다. 어제 배운 `resample`은 구간 안에 값이 하나라도 있으면 평균을 내주므로, 빈 칸이 나왔다는 것은 그 구간이 통째로 비었다는 뜻입니다.

5-3. 실습 2 — 측정 간격 진단

패턴 `index.to_series().diff().dt.total_seconds()` 이웃한 시각의 간격을 초로

정의 인덱스를 Series로 바꾼 뒤 `diff()`로 차이를 구하고, `.dt.total_seconds()`로 초 단위 숫자로 변환합니다.

형태 `df.index.to_series().diff().dt.total_seconds()`

```
gaps = df.index.to_series().diff().dt.total_seconds()

print(gaps.value_counts().sort_index().to_string())
print("최대 간격(초):", gaps.max())
```

```
▶ 3.0      1
▶ 5.0      2
▶ 7.0      3
▶ 9.0      1
▶ 10.0     169
▶ 12.0     1
▶ 13.0     1
▶ 14.0     1
▶ 17.0     1
▶ 20.0     5
▶ 21.0     1
▶ 80.0     1
▶
▶ 최대 간격(초): 80.0
```

왜 쓰나 숫자 분포 하나로 데이터의 성격이 한눈에 드러납니다. 10초가 169개로 압도적이니 "기본 주기는 10초인데 일부 불일치가 섞였다"고 읽습니다. 여러 숫자가 비슷했다면 매우 불규칙한 데이터입니다.

응용 인덱스는 Series가 아니라 Index 객체라 `.diff()`나 `.dt`를 바로 쓸 수 없습니다. `to_series()`로 변환하는 단계가 반드시 필요합니다.

```
# x 인덱스에 바로 .dt를 쓰면 오류
# df.index.diff().dt.total_seconds()

# ✓ to_series()를 거쳐야 합니다
gaps = df.index.to_series().diff().dt.total_seconds()

# 기본 주기를 자동으로 고르기
기준주기 = gaps.mode()[0]
print("가장 흔한 간격:", 기준주기, "초")
print("정상 비율:", round((gaps == 기준주기).mean() * 100, 1), "%")
```

```
▶ 가장 흔한 간격: 10.0 초
▶ 정상 비율: 89.9 %
```

개념

`.mode()`는 **가장 흔한 값**을 돌려주는데, 결과가 Series라 `[0]`으로 첫 값을 꺼내야 합니다. 여러 값이 똑같이 흔하면 여러 개가 나옵니다. **가장 흔한 간격을 기준 주기로 정하는 것이 정규화의 출발점**입니다.

설비 적용

80초가 한 번 있습니다. 10초 주기인데 80초가 비었다면 그 사이 **7개 시점이 통째로 빠진** 것입니다. 센서 단절로 추정되며, 만약 그 70초 사이에 진동 급등이 있었다면 **그 데이터는 영영 사라진** 것입니다. 측정 주기 정리가 "있으면 좋은" 작업이 아니라 **"안 하면 고장을 놓치는" 필수 작업**인 이유입니다.

5-4. 불일치의 두 형태

형태		
모습	10초가 9·11·12초로 조금씩 어긋남	10초가 80초로 벌어짐
상태	값은 있지만 시각이 부정확	여러 시점이 통째로 빔
오늘 데이터	3·5·7·9·12·13·14·17·20·21초 (17건)	80초 (1건)
대처	기준 주기로 재정렬하면 해결	빠진 값을 추정해 보완

설비 적용

현장에서 주기가 어긋나는 이유는 세 가지입니다. 통신 지연으로 전송이 밀리면 10초가 13초가 되고, 센서 단절이면 구간 전체가 비고, 시스템 부하가 걸리면 기록 시각이 미세하게 흔들립니다.

원인을 막을 수는 없지만 **결과는 보정할 수 있습니다.** 흐트러짐을 탓하기보다 원인을 이해하고 후처리로 바로잡는 것이 분석가의 역할입니다.

5-5. 주기가 흐트러지면 생기는 두 위험

위험	무슨 일	결과
① 변화 속도 오독	10초 상승과 80초 상승을 구분 못 함	급격한 이상과 완만한 변화가 같아 보임
② 신호 누락	이상이 나타난 순간 데이터가 통째로 빔	예측 자체가 불가능

주의

데이터가 시간순으로 정렬됐다는 것과 간격이 일정하다는 것은 전혀 다른 이야기입니다. 오늘 데이터는 완벽히 정렬되어 있지만 간격은 3초에서 80초까지 제각각입니다. `sort_index()`를 했다고 안심하면 안 되고, **반드시 간격을 따로 진단**해야 합니다.

STEP 6

asfreq로 격자 정규화

asfreq vs resample · 결측 위치

진단이 끝났으니 **일정한 10초 격자를 만들고 기존 값을 올립니다**. 그 순간 빠져 있던 시점이 NaN으로 드러납니다.

6-1. 실습 3 — asfreq로 정규화

메서드 `.asfreq()` 정해진 주기 격자에 값을 그대로 올리기

정의 격자를 만들고 **시각이 정확히 맞는 칸에만 기존 값을 배치**합니다. 합치거나 계산하지 않으므로 집계 함수가 필요 없습니다.

형태 `df[cols].asfreq("10s")` · `df[cols].asfreq("10s", method="ffill")`

```
cols = ["제어출력", "소입로온도"]
norm = df[cols].asfreq("10s")

print("총 행:", len(norm))
print("결측:", norm.isna().sum().to_dict())

# method 옵션으로 격자를 만들며 동시에 채우기
filled = df[cols].asfreq("10s", method="ffill")
print("ffill 후:", filled.isna().sum().to_dict())
```

```
▶ 총 행: 200
▶ 결측: {'제어출력': 22, '소입로온도': 22}
▶ ffill 후: {'제어출력': 0, '소입로온도': 0}
```

왜 쓰나 188행이 200행이 되면서 22칸이 비었습니다. 격자를 만들지 않았다면 **빠진 시점의 행 자체가 없어** 무엇이 없는지도 몰랐을 것입니다. `asfreq`는 빈 칸을 만드는 것이 아니라 **원래 비어 있던 시점을 드러내는** 도구입니다.

응용 `method="ffill"`이나 `"bfill"`을 주면 **격자를 만들면서 동시에 채웁니다**. 옵션 하나로 NaN을 0개로 만들 수 있습니다.

```
# 옵션 없이 — 빈 칸을 남겨 두고 나중에 정할 때
a = df[cols].asfreq("10s")

# ffill — 앞값을 그대로 끌어와 채움
b = df[cols].asfreq("10s", method="ffill")

# bfill — 뒷값을 앞으로 끌어와 채움
c = df[cols].asfreq("10s", method="bfill")

print([int(x["제어출력"].isna().sum()) for x in [a, b, c]])
```

▶ [22, 0, 0]

개념

NaN은 오류가 아니라 정보입니다. "여기에 원래 값이 없었다"는 정직한 표시입니다. 임의 값으로 슬그머니 채우면 **실제 측정값과 추정값이 섞여 구분이 불가능**해집니다. NaN으로 남겨두기 때문에 신중하게 채울 방법을 고를 수 있습니다.

주의

method 옵션은 **빠른 길**이고 STEP 7의 보간은 **정교한 길**입니다. 앞뒤 값을 단순 복사하므로 천천히 변하는 데이터에는 무난하지만, 빠르게 변하는 데이터에는 부정확할 수 있습니다.

6-2. asfreq와 resample의 차이

구분		
동작	격자에 그대로 놓기	구간으로 묶어 합치기
집계 함수	필요 없음	반드시 필요
어긋난 값	버려짐	평균에 포함됨
원본 보존	그대로 살림	평균으로 바뀜
적합한 방향	업샘플링 (쭈뼌하게)	다운샘플링 (듬성하게)
오늘 결측	22개	16개

```
# 같은 10초 정규화인데 결측 수가 다릅니다
a = df[["제어출력"]].asfreq("10s").rename(columns={"제어출력": "asfreq"})
r = df[["제어출력"]].resample("10s").mean().rename(columns={"제어출력": "resample"})

print(a.join(r).head(8).round(3))
print("asfreq:", int(a["asfreq"].isna().sum()),
      "/ resample:", int(r["resample"].isna().sum()))

# 1분으로 다운샘플링하면 크게 줄어듭니다
down = df[["제어출력"]].resample("1min").mean()
print("1분 다운샘플링:", int(down["제어출력"].isna().sum()))
```

```
▶ asfreq: 22 / resample: 16
▶ 1분 다운샘플링: 1
```

개념

차이가 나는 이유는 어긋난 값을 어떻게 다루느냐입니다. `asfreq`는 격자의 정확한 시각에 값이 없으면 `NaN`이고 09:00:07 같은 어긋난 값은 버려집니다. `resample`은 09:00:00~09:00:09 구간 안에 값이 하나라도 있으면 평균을 채웁니다.

그래서 같은 10초 정규화인데 **asfreq 22개, resample 16개**로 걸립니다. 어느 쪽이 맞는지가 아니라 **목적에 따라** 선택하는 것입니다. 원본 값 하나하나가 중요하다면 `asfreq`, 노이즈를 줄이고 싶으면 `resample`입니다.

1분으로 다운샘플링하면 결측이 1개로 줄어듭니다. 구간이 넓어져 값이 하나라도 걸릴 확률이 높아지기 때문입니다. 다만 해상도가 6분의 1로 떨어집니다.

주의

다운샘플링에 `asfreq`를 쓰면 안 됩니다. 1초 데이터를 1분으로 줄이면서 `asfreq("1min")`를 쓰면 격자의 정확한 시각 값 하나만 가져오고 나머지 59초를 통째로 버립니다. 데이터를 심하게 잃는 부적절한 선택입니다. 헛갈리면 "촘촘하게 만들려는가, 듬성하게 만들려는가" 한 질문으로 판단하십시오. 촘촘하게는 `asfreq`, 듬성하게는 `resample`입니다.

6-3. 실습 4 — 결측의 개수·비율·위치

① 개수
`isna().sum()`



② 비율
`isna().mean()`



③ 위치
`index` 추출

Practice/22_01_example.ipynb — 실습 4

```
norm = df[["제어출력", "소입로온도"]].asfreq("10s")

print("결측 개수:", norm.isna().sum().to_dict())
print("결측 비율(%):", (norm.isna().mean() * 100).round(1).to_dict())

miss = norm[norm["제어출력"].isna()].index
print("빈 시각 개수:", len(miss), "/ 첫 구간:", miss[0])
```

```
▶ 결측 개수: {'제어출력': 22, '소입로온도': 22}
▶ 결측 비율(%): {'제어출력': 11.0, '소입로온도': 11.0}
▶ 빈 시각 개수: 22 / 첫 구간: 2024-03-01 09:02:30
```

설비 적용

11%는 보간이 가능한 수준입니다. 교안 기준으로 50%가 넘으면 그 센서 자체를 의심해야 합니다. 그리고 단순 개수를 넘어 **최초로 유실된 시점(09:02:30)**을 알아내면, 당시 오프라인 정비나 전력 다운 같은 운영 기록과 대조해 원인을 찾을 수 있습니다.

6-4. 실습 6 — 산발 결측과 연속 결측

빈 칸 개수보다 분포 패턴이 처리 전략의 기준입니다. 22개가 흩어져 있는지 한군데 몰려 있는지가 완전히 다른 문제입니다.

STEP 4에서 배운 덩어리 관용구가 여기서 다시 쓰입니다

```
# 연속으로 이어진 결측 길이를 구하기

isna = norm["제어출력"].isna()
groups = (isna != isna.shift()).cumsum()

print("가장 긴 연속 결측(칸):", int(isna.groupby(groups).sum().max()))

runs = isna.groupby(groups).sum()
print("연속 결측 길이:", sorted(runs[runs > 0].tolist(), reverse=True))
```

```
▶ 가장 긴 연속 결측(칸): 10
▶ 연속 결측 길이: [10, 7, 1, 1, 1, 1, 1]
```

패턴		
모습	한 개씩 흩어진 빈 칸	한 구간이 통째로 빈
앞뒤 값 거리	가까움 (10초 전후)	멀리 떨어짐 (100초)
복원 신뢰도	높음	낮음
오늘 데이터	1칸짜리 5개	10칸 · 7칸
처리 방향	보간으로 안전하게	제한 보간 또는 공백 유지

설비 적용

22개의 결측이 7덩어리로 나뉩니다. 그중 10칸(100초)과 7칸(70초)이 통째로 빈 구간이고, 나머지 5개는 1칸짜리 산발 결측입니다.

1칸짜리는 앞뒤 10초 전후 값이 있으므로 **중간값을 꽤 정확히 추정**할 수 있습니다. 하지만 **10칸 연속 구간은 앞뒤가 100초나 떨어져** 있어, 그 사이에 진동 급등이 있었어도 포착할 방법이 없습니다.

그래서 **처리 방법이 달라야 합니다**. 산발 결측은 보간으로 채우고, 긴 연속 구간은 **비워 두거나 추정값임을 명확히 표시**해야 합니다. STEP 7의 limit 옵션이 정확히 이 문제를 다룹니다.

"22개가 비었다"는 숫자 위에 어떻게 흩어졌는지

```
# 결측 위치를 그래프로 — 점과 선을 함께 그려야 빈 구간이 보입니다
plt.figure(figsize=(10, 3))
plt.plot(norm.index, norm["제어출력"], marker=".")
plt.title("제어출력 after 10s asfreq (gaps = missing)")
plt.tight_layout()
plt.show()
```

▶ 선이 끊긴 두 구간이 눈에 보입니다 — 10칸과 7칸 구간

주의

`marker="."`을 반드시 넣으십시오. **선만 그리면 결측 구간이 그냥 직선으로 이어져 보여** 어디가 비었는지 알 수 없습니다. 점을 함께 찍으면 **점이 없는 구간이 빈 곳**이라는 것이 분명해집니다.

STEP 7

보간과 전처리 파이프라인

ffill · bfill · time 보간 · limit · 함수화

격자를 만들고 빈 칸을 드러냈으니 이제 **채울 차례**입니다. 다만 **전부 채우는 것이 항상 옳지는 않습니다.**

7-1. 결측 처리의 두 갈래

방식	원리	메서드	적합한 값
값 복사	앞이나 뒤의 값을 그대로 가져옴	<code>.ffill()</code> <code>.bfill()</code>	설정값 · 모드처럼 유지되는 값
값 계산	앞뒤 실제 값을 함께 반영해 추정	<code>.interpolate()</code>	온도처럼 꾸준히 변하는 값

주의

빈 행을 `dropna()`로 삭제하는 것은 위험합니다. 빈 칸이 사라져도 시간 간격이 다시 불규칙해져 **시간 축 품질이 오히려 악화**됩니다. 애써 만든 10초 격자가 무너지는 셈입니다. 시계열에서는 삭제보다 복원이 원칙입니다.

7-2. 실습 1 — ffill과 bfill

Practice/22_02_example.ipynb — 실습 1

```
norm = df[["제어출력", "소입로온도"]].asfreq("10s")

print("ffill      :", int(norm["제어출력"].ffill().isna().sum()))
print("bfill      :", int(norm["제어출력"].bfill().isna().sum()))
print("ffill+bfill:", int(norm["제어출력"].ffill().bfill().isna().sum()))

▶ ffill      : 0
▶ bfill      : 0
▶ ffill+bfill: 0
```

구분		
가져오는 값	앞쪽 마지막 실제 값	뒤쪽 첫 실제 값
채우는 방향	위에서 아래	아래에서 위
남는 빈 칸	맨 앞 공백	맨 뒤 공백

구분		
오늘 결과	0 (첫 행에 값이 있음)	0 (마지막 행에 값이 있음)

개념

두 방법은 방향만 반대인 값 복사 전략입니다. 오늘 데이터는 첫 행과 마지막 행에 값이 있어 둘 다 0이 나왔지만, 맨 앞이 비어 있으면 `ffill`은 그 칸을 못 채웁니다. 앞에 가져올 값이 없기 때문입니다. 그래서 `ffill().bfill()`처럼 이어 붙여 쓰면 양 끝까지 완전히 채울 수 있습니다.

자주 하는 실수

`ffill`은 추세를 감춥니다. 제어출력이나 온도처럼 시간에 따라 서서히 상승하는 값이라면, 빈 구간이 평평한 계단으로 변형됩니다. 실제로는 계속 오르고 있었는데 그래프에서는 멈춰 있는 것처럼 보이게 됩니다.

그래서 `ffill`은 설정값·가동 모드처럼 유지되는 값이나 한두 칸짜리 짧은 공백에 적합하고, 변화하는 센서값에는 보간이 낫습니다.

7-3. 실습 2 — 선형 보간과 시간 보간

키워드 `method="time"` 실제 시간 간격을 반영한 보간

정의 기본 `interpolate()`는 칸의 개수를 기준으로 하고, `method="time"`은 실제 시간 간격을 기준으로 합니다.

형태 `series.interpolate()` `series.interpolate(method="time")`

```
import numpy as np

# 불규칙 간격 예시: 앞에서 10초, 뒤까지 50초 떨어진 빈 칸
s = pd.Series([10.0, np.nan, 40.0],
              index=pd.to_datetime(["2024-01-01 09:00:00",
                                    "2024-01-01 09:00:10",
                                    "2024-01-01 09:01:00"]))

print("선형 보간:", s.interpolate(method="linear").iloc[1])
print("시간 보간:", s.interpolate(method="time").iloc[1])
```

▶ 선형 보간: 25.0

▶ 시간 보간: 15.0

왜 쓰나 선형은 "가운데 칸"이라 25를 주고, 시간은 "전체 60초 중 10초 지점"이라 15를 줍니다. 앞 값에 훨씬 가까운 시각이므로 시간 보간이 물리적으로 정확합니다.

응용 격자가 완벽히 등간격이면 두 결과가 같습니다. 오늘 데이터는 이미 10초 격자로 정규화했으므로 차이가 없습니다.

```
# 정규화된 데이터에서는 두 결과가 같습니다
lin = norm["제어출력"].interpolate()
tim = norm["제어출력"].interpolate(method="time")
print("선형과 시간이 다른 칸 수:", int((lin != tim).sum()))
```

▶ 선형과 시간이 다른 칸 수: 0

개념

method="time"은 시간 인덱스가 있어야만 동작합니다. 일반 정수 인덱스에 쓰면 오류가 납니다. 반면 선형 보간은 별도 조건이 없습니다. 정규화된 등간격 데이터라면 선형 보간도 안정적인 선택입니다.

주의

그럼에도 method="time"을 습관적으로 쓰는 편이 안전합니다. 정규화를 빠뜨렸거나 격자가 완전하지 않은 경우에도 올바른 값을 주기 때문입니다. 등간격이면 어차피 같은 결과이므로 손해가 없습니다.

7-4. 실습 3 — limit으로 채우기 범위 제한

여기가 오늘 문서에서 가장 중요한 부분입니다. 긴 누락 구간에 보간을 그대로 적용하면 앞뒤 직선이 급등락을 매끈하게 덮어 버립니다.

Practice/22_02_example.ipynb — 실습 3

```
isna = norm["제어출력"].isna()
groups = (isna != isna.shift()).cumsum()
runs = isna.groupby(groups).sum()

print("연속 결측:", sorted(runs[runs > 0].tolist(), reverse=True))
print("제한 없음:", int(norm["제어출력"].interpolate().isna().sum()))
print("두 칸 제한:", int(norm["제어출력"].interpolate(limit=2).isna().sum()))
```

▶ 연속 결측: [10, 7, 1, 1, 1, 1, 1]

▶ 제한 없음: 0

▶ 두 칸 제한: 13

키워드 limit= 연속으로 채울 최대 칸 수 제한

정의 연속된 결측을 최대 몇 칸까지만 채울지 정합니다. 그 이상 이어진 구간은 일부만 채우고 나머지는 NaN으로 남습니다.

형태 `series.interpolate(limit=2)`

```
# limit=2면 10칸 구간에서 앞 2칸만 채우고 8칸을 남깁니다
print("limit 없음:", int(norm["제어출력"].interpolate().isna().sum()))
print("limit=2   :", int(norm["제어출력"].interpolate(limit=2).isna().sum()))
print("limit=5   :", int(norm["제어출력"].interpolate(limit=5).isna().sum()))
```

```
▶ limit 없음: 0
▶ limit=2   : 13
▶ limit=5   : 7
```

왜 쓰나 연속 10칸(100초)이 통째로 빈 구간을 보간으로 대충 지어내는 것은 위험합니다. 그 사이에 진동 급등이 있었어도 직선이 덮어 버립니다. **비워 두고 점검 알람을 울리는 편이 안전한 판단**입니다.

응용 limit 외에도 방향과 영역을 제어하는 옵션이 있습니다.

```
# limit_direction - 채우는 방향
norm["제어출력"].interpolate(limit=2, limit_direction="both")

# limit_area - 채울 영역
# "inside"   : 실제 값 사이의 안쪽 공백만
# "outside"  : 양 끝 바깥만
norm["제어출력"].interpolate(limit_area="inside")
```

개념

limit=2로 13칸이 남는 계산을 확인해 보면, **10칸 구간에서 8칸, 7칸 구간에서 5칸**이 남아 13칸입니다. 1칸짜리 다섯 개는 limit 2 안에 들어가므로 모두 채워집니다. **짧은 안쪽 공백만 골라 채우고 긴 구간은 남기는** 정확히 원하는 동작입니다.

설비 적용

보간을 피해야 할 상황이 세 가지입니다. ① 고장 전조가 직선 보간에 가려질 수 있는 **급변 구간**, ② 설비 정지처럼 **공백 자체가 정보**인 경우, ③ 추정값이 대부분이 되어 신뢰도가 낮아지는 **높은 결측 비율**. 오늘 데이터의 10칸 구간은 ①에 해당할 수 있으므로 남기는 것이 맞습니다.

7-5. 실습 4 — 채운 자리를 표시하기

Practice/22_02_example.ipynb — 실습 4

```
print("복원 전:", norm.isna().sum().to_dict())
restored = norm.interpolate(method="time")
print("복원 후:", restored.isna().sum().to_dict())

# 보간 전에 결측이었던 위치를 기억
was_na = norm["제어출력"].isna()

fig, ax = plt.subplots(2, 1, figsize=(10, 5), sharex=True)
ax[0].plot(norm.index, norm["제어출력"], marker=".")
ax[0].set_title("before")

ax[1].plot(restored.index, restored["제어출력"], marker=".")
# 원래 NaN이었던 자리만 빨간 점으로 표시
ax[1].plot(restored.index[was_na], restored["제어출력"][was_na],
           "ro", markersize=4)
ax[1].set_title("after (red = estimated)")

plt.tight_layout()
plt.show()
```

▶ 복원 전: {'제어출력': 22, '소입로온도': 22}

▶ 복원 후: {'제어출력': 0, '소입로온도': 0}

개념

보간 전에 `was_na`로 결측 위치를 기억해 두는 것이 핵심 기법입니다. 보간하고 나면 어느 값이 실제 측정값이고 어느 값이 추정값인지 구분할 방법이 사라지기 때문입니다. 리포트에 그래프를 넣을 때 추정값을 빨간 점으로 표시하면, 보는 사람이 "이 구간은 추정이구나"를 즉시 알 수 있습니다. 이것이 신뢰받는 리포트의 조건입니다.

7-6. 실습 5·6 — 파이프라인 함수로 묶기

패턴 `preprocess()` 함수 전처리 단계를 하나로 묶어 재사용

정의 불러오기 → 시간 인덱스 → 격자 정규화 → 보간을 **한 함수로 묶어** 파일 경로만 넣으면 깨끗한 데이터가 나오게 만듭니다.

형태 `def preprocess(path, freq="10s"): ... return clean`

```
def preprocess(path, freq="10s"):
    d = pd.read_csv(path)
    d["timestamp"] = pd.to_datetime(d["timestamp"])
    d = d.set_index("timestamp").sort_index()
    g = d[["제어출력", "소입로온도"]].asfreq(freq)

    print("전:", g.isna().sum().to_dict())
    clean = g.interpolate(method="time")
    print("후:", clean.isna().sum().to_dict())

    return clean

clean = preprocess("Data/22_열처리.csv")
```

▶ 전: {'제어출력': 22, '소입로온도': 22}
▶ 후: {'제어출력': 0, '소입로온도': 0}

왜 쓰나 공장에서 센서 CSV가 매일 아침 새로 수집되는 상황을 생각하면, 매번 다섯 줄을 타이핑하는 대신 **파일 경로만 넣으면** 됩니다. 일관성·재현성·실수 방지를 동시에 얻습니다.

응용 freq를 인자로 받았으므로 **주기를 바꿔 가며 비교**할 수 있습니다.

```
for freq in ["10s", "30s", "1min"]:
    n = df[["제어출력"]].asfreq(freq)
    c = n["제어출력"].interpolate(method="time")
    print(freq, "행", len(n),
          "결측", int(n["제어출력"].isna().sum()),
          "보간후", int(c.isna().sum()))
```

▶ 10s 행 200 결측 22 보간후 0
▶ 30s 행 67 결측 9 보간후 0
▶ 1min 행 34 결측 5 보간후 0

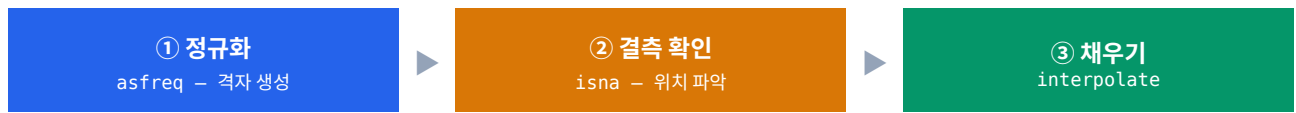
개념

주기가 길수록 행 수와 결측이 함께 줄어듭니다. 10초는 200행에 22결측, 1분은 34행에 5결측입니다. 다만 해상도가 낮아지므로 **짧은 이상 신호를 놓칠 위험**이 커집니다. 고해상도로 같지 가벼운 저해상도로 같지는 분석 목적에 따라 정합니다.

자주 하는 실수

함수 안에서 return을 빠뜨리는 것이 가장 흔한 실수입니다. print로 결과가 보여도 함수 밖에서는 None이 반환되어 다음 줄에서 AttributeError가 납니다.

7-7. 순서가 만드는 차이



순서	동작	결과
정규화 → 보간	격자 생성 후 빈 칸 채움	일정 간격 + 결측 복원
보간 → 정규화	빈 행이 드러나기 전에 처리	채우기 실패 + 간격 불규칙

주의

정규화하지 않은 원본은 빠진 시각의 행 자체가 없습니다. 그래서 interpolate()를 먼저 부르면 채울 대상이 없어 아무 일도 일어나지 않습니다. 오늘 데이터의 원본 결측이 0이었던 것을 떠올리면 이해가 쉽습니다. 격자를 만들어 "여기 칸이 비었다"를 드러낸 뒤에야 보간이 작동합니다. 보간의 전제는 명확한 시간 격자와 빈 칸 위치입니다.

7-8. 전처리 품질 점검 체크리스트

점검 항목	확인 내용	통과 기준	오늘 결과
시간 간격	일정한 간격 유지	모든 시점 동일 간격	10초 격자 완성
결측 처리	남은 NaN 상태	0개 또는 의도한 수	0개 (또는 limit 13)
추세 보존	전후 그래프 비교	왜곡 없는 흐름	계단·점프 없음
신뢰도 표시	긴 보간 구간 기록	추정 구간 명시	추정 비율 11.0%
전체 품질	결측 비율 수준	과도 시 제외 검토	11%는 허용 범위

7-9. 실습 7 — 정비 판단으로 잇기

Practice/22_02_example.ipynb — 실습 7

```
# 복원한 데이터로 추세를 계산하고 상태를 분류

clean_v = norm["제어출력"].interpolate(method="time")
trend = clean_v.rolling(window=6, min_periods=1).mean()

final = trend.iloc[-1]
change = trend.iloc[-1] - trend.iloc[0]

status = ("정비 검토 필요" if (final > 0.55 or change > 0.1)
          else ("주의" if change > 0.05 else "정상"))

print("최종 추세:", round(final, 3), "변화량:", round(change, 3))
print("판단:", status)
print("추정값 비율(%):", round(norm["제어출력"].isna().mean() * 100, 1))
```

▶ 최종 추세: 0.8 변화량: 0.374

▶ 판단: 정비 검토 필요

▶ 추정값 비율(%): 11.0

키워드 min_periods= 창이 다 안 차도 계산하기

정의 rolling()에서 최소 몇 개만 있으면 계산할지 정합니다. 기본값은 window와 같아서 앞쪽이 NaN이 됩니다.

형태 series.rolling(window=6, min_periods=1).mean()

```
s = pd.Series([1, 2, 3, 4, 5])

print("기본 :", s.rolling(3).mean().round(2).tolist())
print("min_periods=1:", s.rolling(3, min_periods=1).mean().round(2).tolist())
```

▶ 기본 : [nan, nan, 2.0, 3.0, 4.0]

▶ min_periods=1: [1.0, 1.5, 2.0, 3.0, 4.0]

왜 쓰나 min_periods=1을 주면 앞쪽 NaN이 사라집니다. 첫 값은 자기 자신, 두 번째는 두 개 평균으로 계산됩니다.

그래서 trend.iloc[0]을 바로 쓸 수 있습니다.

응용 다만 앞쪽 값들은 신뢰도가 낮습니다. 하나나 둘로 낸 평균이라 창이 짝 찬 뒤와 성격이 다릅니다.

```
# 앞쪽 값이 몇 개로 계산됐는지 확인
cnt = clean_v.rolling(6, min_periods=1).count()
print(cnt.head(8).tolist())
```

```
▶ [1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 6.0, 6.0]
```

개념

27일차에서 `rolling(20)`이 앞 19칸을 비웠던 것과 대조됩니다. 어느 쪽이 맞다기보다 목적에 따라 고릅니다. 앞 구간을 버려도 되면 기본값, 첫 값부터 필요하면 `min_periods=1`입니다.

주의

`min_periods=1`을 쓸 때는 **몇 개로 계산됐는지 함께 확인**하는 것이 좋습니다. `.count()`를 붙이면 각 시점의 유효 개수가 나옵니다.

설비 적용

최종 추세 0.8, 변화량 +0.374로 "정비 검토 필요"가 나왔습니다. 제어출력이 34분 동안 0.43에서 0.8까지 올라갔다는 뜻입니다.

중요한 것은 **"추정값 비율 11.0%"를 함께 출력**한다는 점입니다. 판단 근거의 11%가 실제 측정이 아니라 보간값이라는 사실을 밝히는 것입니다. **데이터 품질까지 함께 해석**해야 신뢰받는 리포트가 됩니다.

그리고 이 임계값들(0.55, 0.1, 0.05)은 누군가 정해 준 것이 아니라 이 설비에 맞게 정한 값입니다. 다른 설비에서는 다른 값을 써야 합니다. 화재경보기처럼 너무 민감하면 거짓 경보, 너무 둔감하면 진짜를 놓치는 균형의 문제입니다.

A-1. 전처리 파이프라인 전체 코드

여섯 단계를 한 함수로

```
import pandas as pd

def preprocess(path, freq="10s", cols=None, limit=None):
    # ① 시간 인덱스
    d = pd.read_csv(path)
    d["timestamp"] = pd.to_datetime(d["timestamp"])
    d = d.set_index("timestamp").sort_index()

    # ② 간격 진단
    gaps = d.index.to_series().diff().dt.total_seconds()
    print("기준 주기:", gaps.mode()[0], "초 | 최대 간격:", gaps.max(), "초")

    # ③ 정규화
    g = d[cols].asfreq(freq)

    # ④ 결측 확인
    print("결측:", g.isna().sum().to_dict())
    print("비율(%):", (g.isna().mean() * 100).round(1).to_dict())
    isna = g[cols[0]].isna()
    runs = isna.groupby((isna != isna.shift()).cumsum()).sum()
    print("연속 결측:", sorted(runs[runs > 0].tolist(), reverse=True))

    # ⑤ 채우기
    clean = g.interpolate(method="time", limit=limit)

    # ⑥ 재확인
    print("남은 결측:", clean.isna().sum().to_dict())
    return clean

clean = preprocess("Data/22_열처리.csv", cols=["제어출력", "소입로온도"])
```

▶ 기준 주기: 10.0 초 | 최대 간격: 80.0 초

- ▶ 결측: {'제어출력': 22, '소입로온도': 22}
- ▶ 비율(%): {'제어출력': 11.0, '소입로온도': 11.0}
- ▶ 연속 결측: [10, 7, 1, 1, 1, 1, 1]
- ▶ 남은 결측: {'제어출력': 0, '소입로온도': 0}

A-2. 오늘의 도구

도구	하는 일	예	주의
<code>.diff()</code>	직전 값과의 차	<code>s.diff()</code>	첫 값 NaN
<code>.pct_change()</code>	직전 값 대비 비율	<code>s.pct_change() * 100</code>	0이 있으면 inf
<code>seasonal_decompose()</code>	추세·계절·잔차 분리	<code>(s, "additive", period=24)</code>	결측 있으면 오류
<code>.corr()</code>	센서 쌍 상관행렬	<code>df[sensors].corr()</code>	인과 아님
<code>sns.heatmap()</code>	상관을 색으로	<code>vmin=-1, vmax=1</code>	범위 고정 필수
<code>plt.axvspan()</code>	구간 색칠	<code>(265, 287, alpha=0.15)</code>	alpha 낮게
<code>index.to_series()</code>	인덱스를 Series로	<code>.diff().dt.total_seconds()</code>	변환 필수
<code>.asfreq()</code>	격자에 값 올리기	<code>.asfreq("10s")</code>	집계 안 함
<code>.ffill()</code> <code>.bfill()</code>	앞뒤 값 복사	<code>.ffill().bfill()</code>	추세를 감춤
<code>.interpolate()</code>	앞뒤 이어 채우기	<code>method="time", limit=2</code>	추정값임을 표시

A-3. 헛갈리는 짝 정리

A	B	한 줄 차이	판단 질문
<code>.diff()</code>	<code>.pct_change()</code>	변한 양 vs 변한 비율	크기 다른 센서를 비교하나
<code>.asfreq()</code>	<code>.resample()</code>	격자에 놓기 vs 구간 묶기	촘촘하게인가 듬성하게인가
<code>.ffill()</code>	<code>.interpolate()</code>	복사 vs 계산	유지되는 값인가 변하는 값인가
linear	time	칸 개수 vs 실제 간격	격자가 등간격인가
고정 밴드	이동 밴드	전체 기준 vs 구간 기준	추세가 있는가
axhline	axvline	가로선 vs 세로선	값 기준인가 시점 기준인가

A-4. 결측 처리 선택 기준

상황	선택	이유
설정값 · 가동 모드	<code>.ffill()</code>	값이 유지되는 성격
온도 · 압력처럼 연속 변화	<code>.interpolate()</code>	앞뒤를 이은 값이 자연스러움
한두 칸 짧은 공백	<code>.ffill()</code> 또는 <code>.interpolate()</code>	어느 쪽이든 무난
긴 연속 공백	<code>limit=</code> 또는 공백 유지	추정 신뢰도가 낮음
급변 구간	보간 금지	전조가 직선에 가려짐
결측 비율 50% 초과	센서 제외 검토	추정값이 대부분

개념

보간값은 진짜가 아니라 추정입니다. 짧은 누락에는 보간, 긴 누락에는 제외가 낫습니다. 그리고 **채운 값을 측정값처럼 맹신하면 안 됩니다**. 보간 전에 `was_na`로 위치를 기억해 두고, 리포트에는 추정값 비율을 함께 밝히십시오.

오류·함정 사전

X 이렇게 하면 → ✓ 바로잡기

B-1. 자주 만나는 오류 열두 가지

X 이렇게 하면	무슨 일이	✓ 바로잡기
<code>df.index.diff().dt</code>	Index에는 .dt가 없어 오류	<code>index.to_series().diff().dt</code>
<code>.asfreq("10S")</code>	대문자는 폐기 예정 — 경고	<code>.asfreq("10s")</code>
<code>.resample("1T")</code>	대문자 T도 폐기	<code>.resample("1min")</code>
보간 먼저, 정규화 나중	채울 대상이 없어 아무 일도 안 일어남	정규화 → 보간 순서
다운샘플링에 <code>asfreq</code>	격자 시각 하나만 남고 나머지 버려짐	<code>.resample().mean()</code>
<code>.interpolate(method="time")</code>	시간 인덱스 없으면 오류	인덱스를 먼저 시간으로
<code>seasonal_decompose(s)</code>	결측 있으면 오류	<code>.interpolate()</code>
<code>dropna()</code>	행이 사라져 간격이 다시 불규칙	삭제 대신 보간
함수에서 return 누락	None이 반환되어 다음 줄에서 오류	<code>return clean</code>
<code>sns.heatmap(corr)</code>	색 기준이 데이터마다 달라짐	<code>vmin=-1, vmax=1</code>
<code>rolling(6)</code>	앞 5칸이 NaN이라 <code>iloc[0]</code> 이 NaN	<code>min_periods=1</code>
<code>pct_change()</code>	원래 값에 0이 있으면 inf	<code>.replace([np.inf, -np.inf], np.nan)</code>

B-2. 그림은 맞는데 결론이 틀리는 경우

함정	왜 위험한가	대응
원본 결측 0이면 안심	행 자체가 없어 결측으로 안 잡힘 — 실제 22칸	<code>asfreq</code>
정렬됐으니 간격도 일정	정렬과 등간격은 전혀 다른 이야기 — 3~80초	간격을 따로 진단

함정	왜 위험한가	대응
긴 공백도 보간으로 채움	100초 공백을 직선이 덮음 — 급변이 사라짐	limit=
ffill로 다 채우면 끝	상승 구간이 평평한 계단으로 변형	변하는 값에는 보간
변화량 그래프만 보고 판정	노이즈·운영 변화·측정 오류를 구분 못 함	원본·다른 센서·운영 기록 대조
상관 높으니 원인이다	공통 원인 때문일 뿐	인과 단정 금지
센서 다 넣으면 좋다	$ r > 0.95$ 이상이면 정보가 안 늘어남	다중공선성 확인
전체 평균으로 이상 탐지	추세가 있으면 거의 안 잡힘 — 2개	이동 밴드 사용
분해 계절 성분을 그대로 믿음	period는 우리가 넣은 가정	진폭·비중을 확인

주의

오늘 데이터의 가장 큰 함정은 첫 줄입니다. `df.isna().sum()`이 전부 0이라 "결측 없음"으로 보이지만, 실제로는 22개 시점이 통째로 빠져 있었습니다. 행이 없으면 결측으로도 안 잡힙니다. `asfreq`로 격자에 올려야 비로소 드러납니다. 시계열에서 결측 점검은 `isna()` 하나로 끝나지 않습니다.

실습 하이라이트와 다음 시간

오늘의 결론 · 예고

C-1. 실습 한눈에

교안	실습	한 일	핵심 결과
21_02	1	변화량으로 급변 탐지	s_11 최대 급변 0.54 (177회차)
	2	추세·변동성·변화량 3칸	세 관점을 한 화면에
	3	시간 축으로 추세 보기	192행 · 빠진 시각 0개
	4	6시간·하루 집계	32개 · 8개 · 641.4 → 646.63
	5	ffill vs 보간	[10,10,10,16,18] vs [10,12,14,16,18]
	6	시계열 분해	추세 std 4.370 · 계절 진폭 0.791
21_03	1	상관 heatmap	s_2-s_4 +0.99
	2	연동 묶음 찾기	$ r \geq 0.9$ 인 쌍 6개
	3	의심 구간 색칠	axvspan(265, 287)
	4	정상 범위 이탈	1583.17~1608.94 · 이탈 2개
	5	종합 리포트 + 자동 검출	가장 긴 구간 267~287회차
22_01	1	1분 리샘플링	34행 중 빈 칸 1개
	2	측정 간격 진단	10초 169개 · 최대 80초
	3	asfreq 정규화	188 → 200행 · 결측 22
	4	결측 개수·비율·위치	11.0% · 첫 시각 09:02:30
	5	asfreq vs resample	22 vs 16 · 1분은 1
	6	연속 결측 진단	가장 긴 연속 10칸
22_02	1	ffill·bfill	모두 0
	2	선형 vs 시간 보간	예시 25.0 vs 15.0 · 실데이터 동일
	3	limit으로 제한	제한 없음 0 · limit=2 → 13 남김

교안	실습	한 일	핵심 결과
	4	채운 자리 표시	빨간 점으로 추정값 구분
	5·6	파이프라인 함수	10s/30s/1min — 200/67/34행
	7	정비 판단 리포트	추세 0.8 · 변화 +0.374 → 정비 검토

C-2. 오늘 데이터가 말해 준 것

스물세 개의 실습을 **하나의 흐름**으로 다시 읽어 보겠습니다.

설비 적용

- ① 세 번째 관점을 얻었습니다. 추세와 변동성에 **변화량**이 더해졌습니다. `s_3`의 최대 급변이 236회차, `s_4`가 268회차로 **흩어져 있어** 각각 우연일 가능성이 큼니다. 같은 회차에서 동시에 튀었다면 설비 전체 사건이었을 것입니다.
- ② 주기가 없다는 것도 결론입니다. `seasonal_decompose`를 돌렸더니 계절 성분의 진폭이 0.791로 원본 대비 3.7%뿐이고, 잔차(0.713)가 계절(0.189)보다 훨씬 큼니다. **이 데이터에는 하루 주기 무늬가 사실상 없습니다.** `period=24`는 우리가 넣은 가정이었습니.
- ③ 네 센서가 같은 것을 재고 있었습니다. `s_2·s_3·s_4`가 서로 +0.95~+0.99, `s_7`이 이들과 -0.93~-0.96입니다. 하나만 알면 나머지를 거의 맞힐 수 있는 **다중공선성** 상태입니다. `s_11`만 0.81~0.83으로 추가 정보를 갖습니다.
- ④ 원본 결측 0이 거짓이었습니다. `22_열처리.csv`는 `isna().sum()`이 전부 0인데, 10초 격자에 올리자 **22칸 (11%)**이 드러났습니다. **행이 없으면 결측으로도 안 잡힙니다.**
- ⑤ 채우지 않는 것이 답일 때가 있습니다. 22칸 중 **10칸과 7칸이 연속**입니다. 10초와 7초를 직선으로 덮으면 그 사이의 급변이 사라집니다. `limit=2`로 **13칸을 일부러 남긴 것**이 오늘의 결론입니다.

C-3. 24~28일차 다섯 회차의 연결



회차	배운 것	오늘 어디서 다시 쓰었나
22·23일차	<code>isna()</code> · <code>fillna</code> 계열	STEP 6·7 결측 진단과 복원
24일차	IQR 이상치 경계	정상 범위 $\pm 2\sigma$ 와 같은 발상
25일차	<code>subplots</code> · <code>axhline</code>	3칸 대조 · <code>axvspan</code> 으로 확장
26일차	<code>corr()</code> · <code>heatmap</code>	STEP 3 다중 센서 · <code>vmin/vmax</code> 추가

회차	배운 것	오늘 어디서 다시 쓰였나
27일차	rolling · resample · interpolate	limit · min_periods · asfreq로 확장
28일차	diff · decompose · asfreq · 파이프라인	전처리 6단계 완성

연결

오늘 새로 배운 도구는 많지 않습니다. diff · asfreq · axvspan · seasonal_decompose 정도이고, 나머지는 이미 아는 도구에 옵션을 하나씩 더한 것입니다. interpolate에 limit, rolling에 min_periods, heatmap에 vmin/vmax. 옵션 하나가 결론을 바꾸는 지점을 익힌 날입니다.

C-4. 다음 시간 예고

강사님 말씀대로 23장으로 넘어갑니다. 데이터도 새로 배포되어 있습니다.

순서	교안	새 데이터	예상 내용
① 23_01	상태 변화 추적	23_cmapss_fd001_sample.csv	상태가 바뀌는 시점 추적
② 23_02	다중센서 패턴 분석	23_cmapss_unit1_timestamp.csv	여러 센서의 패턴 종합

개념

오늘 STEP 3~4에서 배운 **다중 센서 관점**과 STEP 5~7의 **전처리 파이프라인**이 그대로 이어질 것입니다. 그리고 새 데이터가 23_cmapss_unit1_timestamp.csv로 **timestamp 버전까지 함께** 배포된 것을 보면, cycle과 timestamp를 오가는 구성이 계속될 것으로 보입니다. **오늘 만든 preprocess()** 함수를 그대로 쓸 수 있게 **준비해 두시면 좋습니다.**

미리보기

다음 시간 준비 — 오늘 함수를 재사용할 수 있게

```
def preprocess(path, freq="10s", cols=None, limit=None):  
    d = pd.read_csv(path)  
    d["timestamp"] = pd.to_datetime(d["timestamp"])  
    d = d.set_index("timestamp").sort_index()  
    g = d[cols].asfreq(freq)  
    return g.interpolate(method="time", limit=limit)
```

새 데이터에도 컬럼 이름만 바꿔 바로 적용

```
# clean = preprocess("Data/23_cmapss_unit1_timestamp.csv",  
#                    freq="1h", cols=["s_2", "s_3", "s_4"])
```

▶ 한 번 만들어 두면 새 파일마다 경로만 바꾸면 됩니다

오늘 배운 것	다음 시간 예상 쓰임
<code>preprocess()</code> 파이프라인	새 데이터를 같은 흐름으로 정제
<code>corr()</code> + heatmap	다중센서 패턴 분석의 기본 도구
<code>axvspan</code> 으로 구간 표시	상태가 바뀌는 시점을 색으로
덩어리 관용구 (<code>s != s.shift()</code>). <code>cumsum()</code>	상태 변화 구간을 나누는 데 그대로
사실과 해석 구분	리포트 작성의 기본 원칙

개념

특히 덩어리 관용구를 눈여겨보십시오. 오늘 STEP 4에서 의심 구간을 나눌 때, STEP 6에서 연속 결측 길이를 잘 때 두 번 나왔습니다. "상태 변화 추적"이라는 23_01의 제목을 보면 이 관용구가 세 번째로 나올 가능성이 큼니다. 값이 바뀌는 지점마다 번호를 새로 매기는 이 패턴은 시계열에서 계속 쓰입니다.

오늘의 성취와 복습 루틴

오늘 할 수 있게 된 것

아래 여섯 가지를 코드를 보지 않고 할 수 있다면 오늘 수업은 소화한 것입니다.

#	할 수 있게 된 것	확인 방법
1	변화량과 변화율을 구분해 쓴다	크기 다른 센서 비교엔 pct_change인 걸 아나
2	센서 간 상관을 heatmap으로 읽는다	vmin=-1, vmax=1이 왜 필요한지 아나
3	의심 구간을 그래프에 표시한다	axvspan의 alpha를 얼마로 줄지 아나
4	측정 간격을 진단한다	to_series()가 왜 필요한지 아나
5	asfreq와 resample을 나눠 쓴다	"촘촘하게인가 듬성하게인가"로 판단하나
6	채우지 말아야 할 구간을 남긴다	limit=2가 왜 13칸을 남기는지 아나

복습 루틴 — 3일 계획

시점	할 일	시간
오늘 저녁	22_열처리.csv로 간격 진단 → asfreq → 결측 확인을 코드 안 보고	25분
내일	preprocess() 함수를 처음부터 다시 작성하고 30초·1분 주기로 비교	30분
주말	2번이 아닌 다른 엔진으로 상관 heatmap을 그려 다중공선성 확인	35분

주의

오늘 배운 것 중 가장 실무적인 것은 limit입니다. 100초가 통째로 빈 구간을 직선으로 채우면 그 사이의 사고가 사라집니다. 어디를 채우지 않을지 정하는 것이 더 어려운 판단입니다.

그리고 마지막 실습이 분석을 정비 판단으로 잇는 것이었습니다. 추세 0.8, 변화량 +0.374에 "추정값 비율 11%"를 함께 붙여야 정직한 리포트가 됩니다. 분석은 조연자이지 결정자가 아닙니다.

수고하셨습니다.